TECHCART

Historias de usuario, sprints, arquitectura y UML

Proyecto integrador - Marketplace tecnológico responsive con Programación Orientada a Objetos (POO), Single Page Application (SPA), persistencia local y chatbot de navegación.

DOCUMENTACIÓN FUNCIONAL Y TÉCNICA

2026

1. Contexto del proyecto

TechCart simula la experiencia completa de una tienda tecnológica sin depender de backend ni pagos reales.

PROBLEMA

Las tiendas de demostración suelen limitarse a una interfaz estática: no validan el acceso, no conservan el carrito, no modelan el dominio con POO y no separan correctamente las responsabilidades del código.

Riesgos principales

- Experiencia poco realista.
- Código difícil de mantener.
- Baja demostración de POO.

PROPUESTA

Marketplace académico con registro e inicio de sesión, panel SPA, catálogo cargado desde JSON, carrito persistente, modificación segura del perfil, órdenes, chatbot y diseño responsive.

🔐 **Acceso**
Registro, login y sesión

🛒 **Compra**
Catálogo + carrito

🧩 **POO**
Modelos, herencia y servicios

💬 **Soporte**
Chatbot predeterminado

📱 **UX**
Responsive en desktop/tablet/móvil

💾 **Persistencia**
JSON + LocalStorage

2. Épica e historia pivote

La épica agrupa el valor global; la historia pivote conecta los flujos más importantes del MVP.

ÉPICA E-01

Experiencia integral de compra tecnológica

Como cliente de TechCart, quiero registrarme, explorar el catálogo, administrar un carrito y generar una orden, para realizar una compra tecnológica completa desde una interfaz clara y segura.

Valor de negocio

- Demuestra el flujo end-to-end.
- Integra POO, DOM, persistencia y UX.

HISTORIA PIVOTE HU-05

Gestionar un carrito persistente

Como cliente autenticado, quiero agregar productos, cambiar cantidades, eliminar artículos y conservar el carrito aunque recargue la página, para controlar mi compra antes de generar la orden.

¿Por qué es pivote?

- Conecta catálogo, sesión, LocalStorage y orden.
- Es el núcleo transaccional del marketplace.

3. Backlog de historias de usuario

Formato: Como [rol], quiero [acción], para [beneficio].

ID	Historia	Descripción	Prioridad
HU-01	**Registro**	Como visitante, quiero crear una cuenta con mis datos para acceder a la tienda.	Must
HU-02	**Inicio de sesión**	Como cliente, quiero iniciar sesión con máximo tres intentos fallidos para proteger mi cuenta.	Must
HU-03	**Perfil seguro**	Como cliente, quiero modificar mis datos y contraseña tras verificar mi clave actual.	Must
HU-04	**Catálogo**	Como cliente, quiero explorar productos por categoría y búsqueda para encontrar lo que necesito.	Must
HU-05	**Carrito**	Como cliente, quiero administrar un carrito persistente para preparar mi compra.	Must
HU-06	**Orden**	Como cliente, quiero confirmar la compra y generar una orden con total y productos.	Must
HU-07	**Historial**	Como cliente, quiero consultar mis compras para revisar órdenes anteriores.	Should
HU-08	**Chatbot**	Como cliente, quiero usar un asistente con respuestas rápidas para navegar y resolver dudas.	Should
HU-09	**Responsive**	Como usuario móvil, quiero usar TechCart sin perder funciones en pantallas pequeñas.	Must
HU-10	**Cerrar sesión**	Como cliente, quiero cerrar sesión y proteger mis datos al terminar.	Must

4. Criterios de aceptación clave

Los criterios hacen verificable cada historia y evitan que una función se considere terminada solo por “verse bien”.

HU-02 · Login

- Valida usuario/correo y contraseña.
- Bloquea después de 3 intentos fallidos.
- Guarda la sesión y redirige al panel.

HU-03 · Perfil

- Campos bloqueados inicialmente.
- Solicita contraseña actual para habilitar edición.
- Actualiza cliente y sesión en LocalStorage.

HU-05 · Carrito

- Agregar incrementa el contador.
- Permite +, -, eliminar y recalcular total.
- Persiste tras recargar la página.

HU-06 · Orden

- Se genera solo con carrito no vacío.
- Conserva fecha, cliente, productos y total.
- Vacía el carrito después de confirmar.

HU-08 · Chatbot

- Botón flotante disponible en la SPA.
- Respuestas predeterminadas según opción.
- Puede navegar a productos, carrito, perfil y compras.

HU-09 · Responsive

- Sidebar colapsa en tablet/móvil.
- Formularios y tarjetas se adaptan.
- No hay scroll horizontal no intencional.

5. Plan de sprints

Estimación académica: 4 sprints de 1 semana (5 días hábiles) = aproximadamente 4 semanas.

SPRINT 1

Acceso y base técnica

5 días

- Estructura del proyecto
- Registro + login
- LocalStorage y sesión
- Perfil básico

Entregable

Usuario autenticado

SPRINT 2

Catálogo y POO

5 días

- Producto + herencia
- JSON de productos
- ProductService
- Búsqueda/categorías

Entregable

Catálogo dinámico

SPRINT 3

Carrito y órdenes

5 días

- CarritoCompra
- Persistencia
- Cantidades y total
- Orden + historial

Entregable

Compra funcional

SPRINT 4

SPA, UX y cierre

5 días

- Router + sidebar
- Chatbot
- Responsive
- QA + documentación

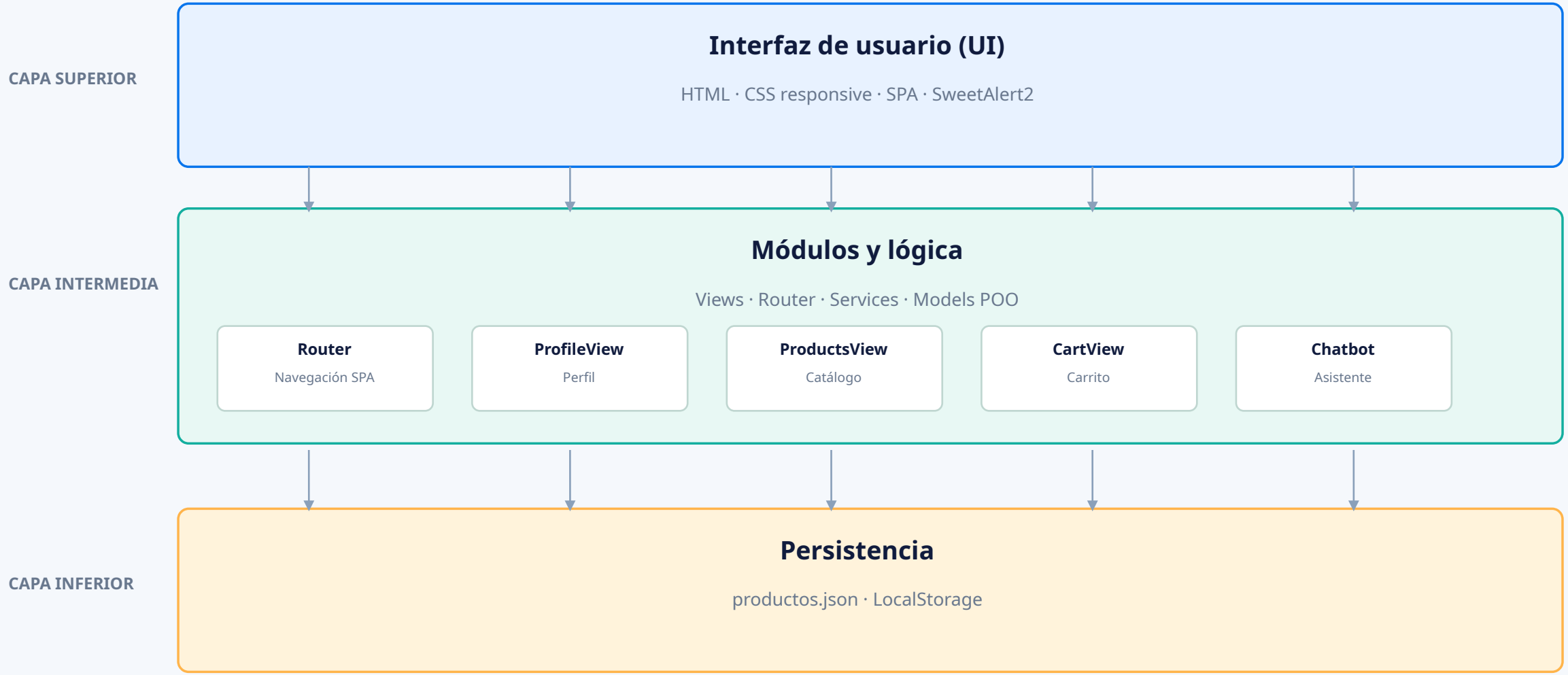
Entregable

Aplicación integrada

Nota: la duración puede variar según el alcance final del curso, pruebas y correcciones solicitadas.

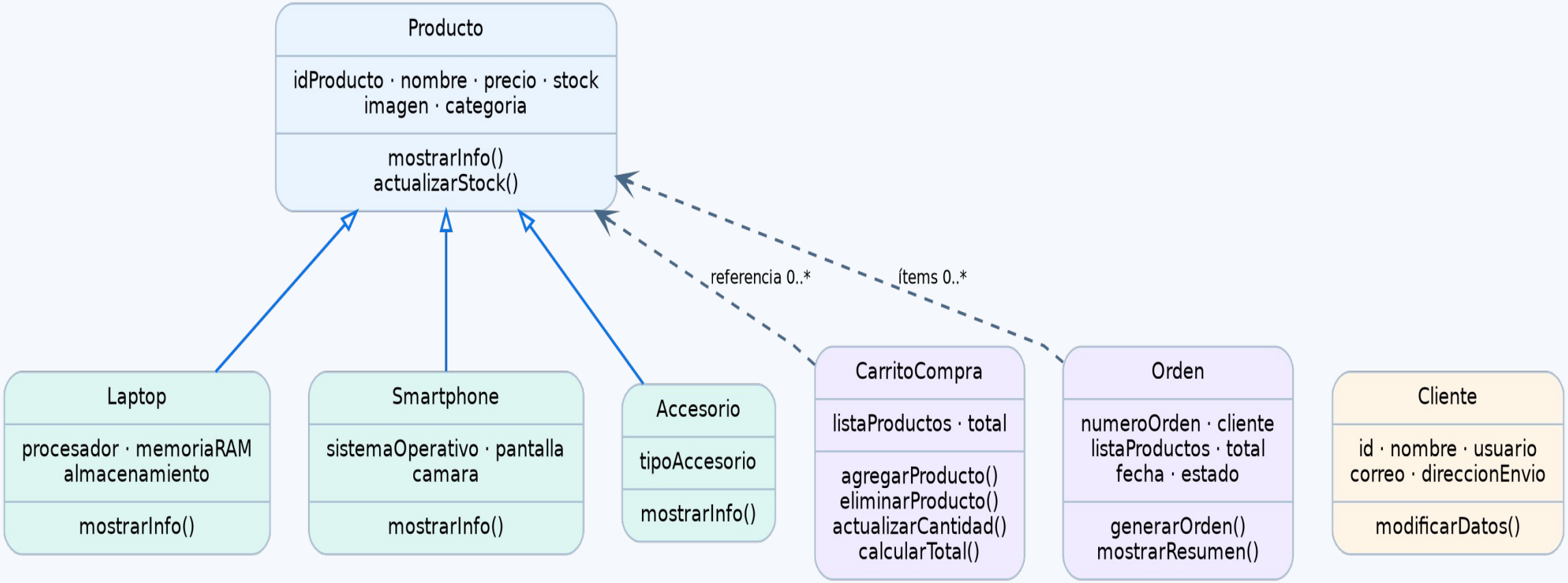
6. Arquitectura de TechCart

Arquitectura modular en tres capas: interfaz SPA, lógica de aplicación/POO y persistencia local.



UML DE CLASES · NÚCLEO CORREGIDO

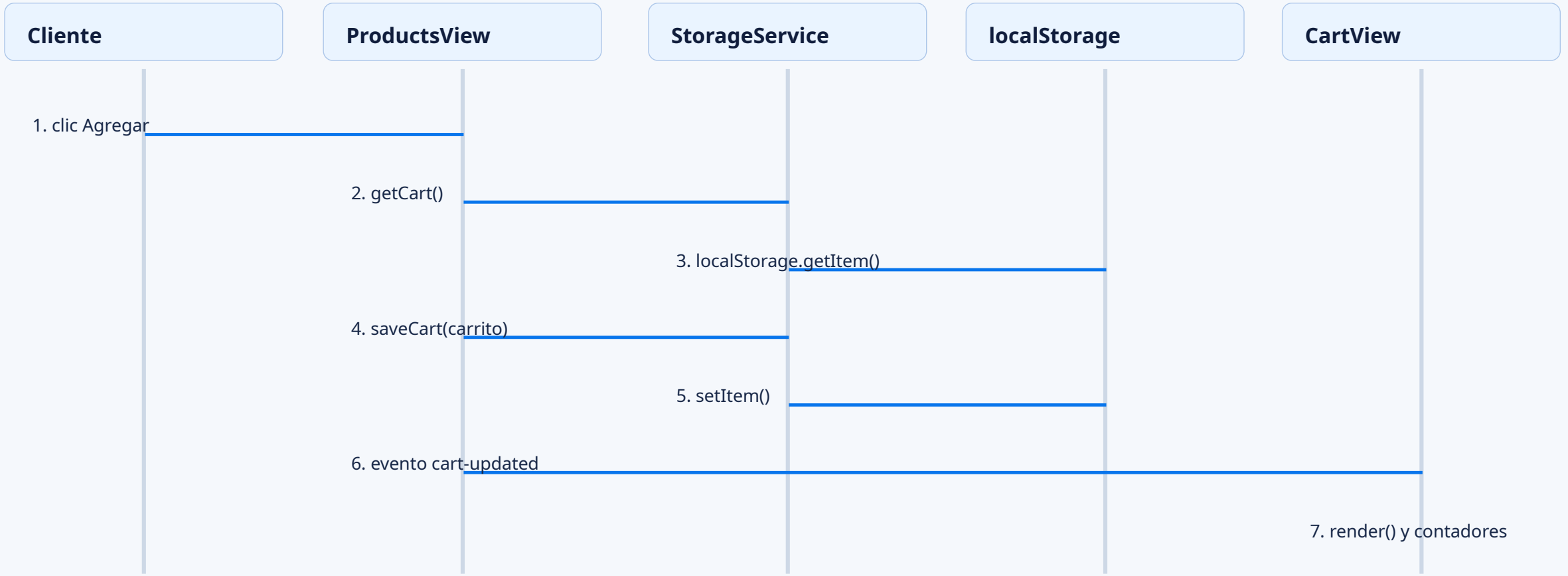
Herencia exclusivamente hacia Producto. Las referencias de objetos no son herencia.



△ **Triángulo hueco: herencia hacia superclase. Línea discontinua: referencia o dependencia.**

SECUENCIA REAL · AGREGAR AL CARRITO

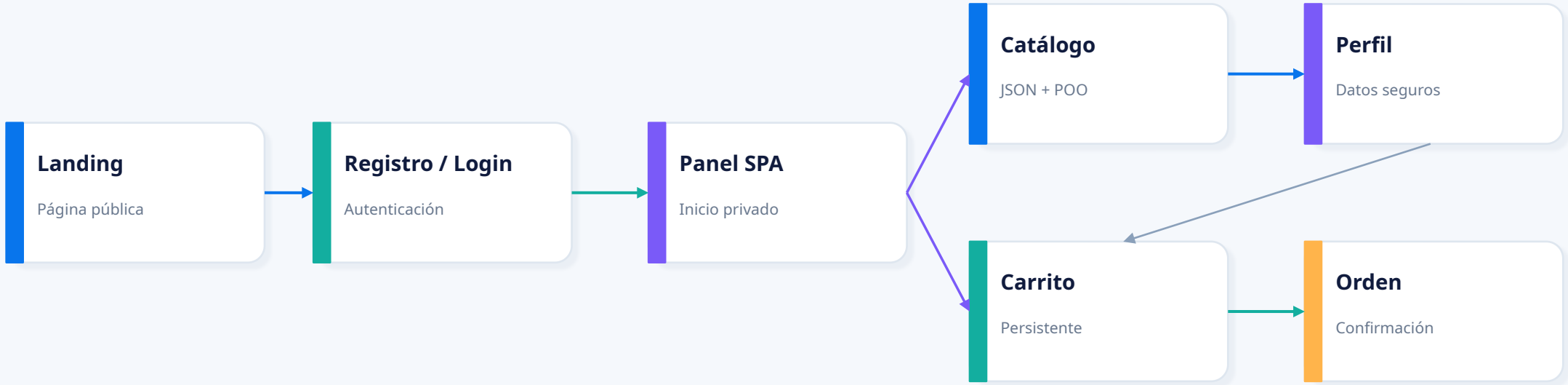
Alineada con ProductsView.js y StorageService.js revisados.



ProductsView modifica arreglo y persiste con StorageService; CartView reacciona al evento.

9. Flujo funcional de la aplicación

El usuario avanza desde la zona pública hasta la compra sin abandonar el mismo ecosistema visual.



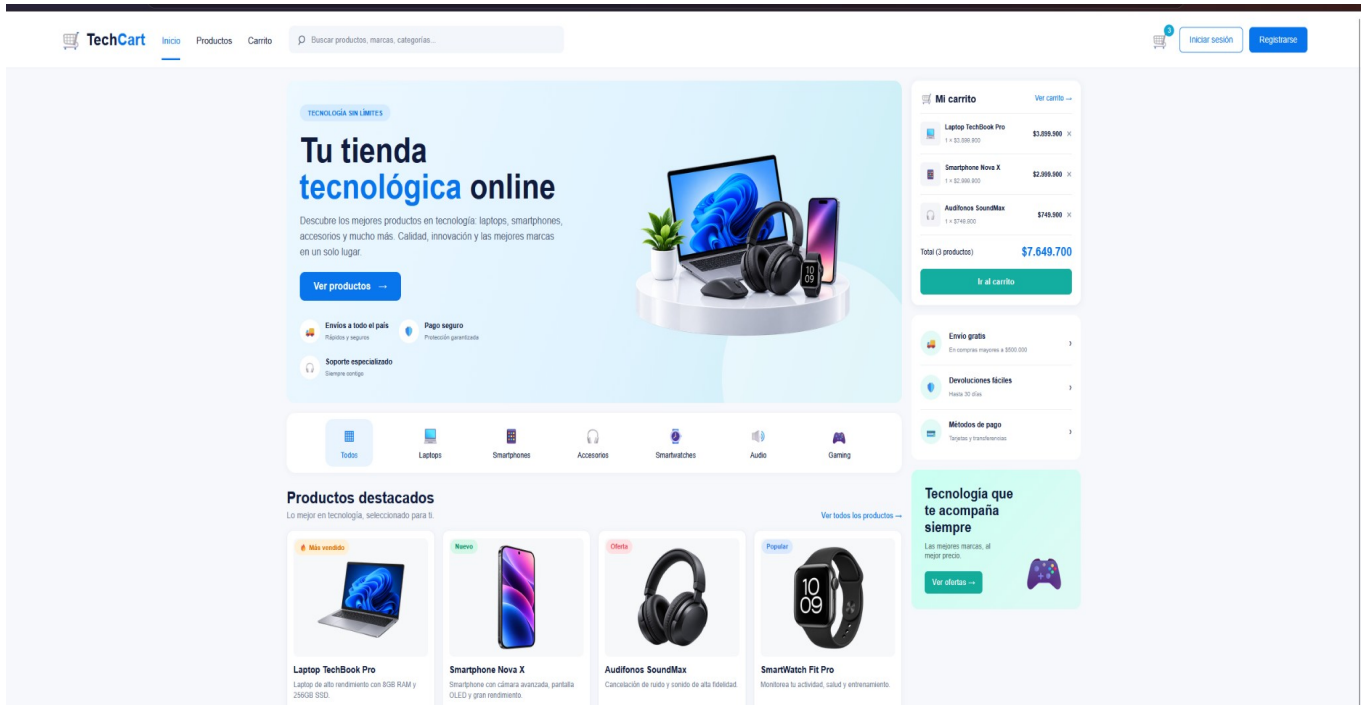
Chatbot disponible en todo el panel: navega, orienta y responde dudas frecuentes.

💬 **TechBot → Productos · Carrito · Compras · Envíos · Pagos · Perfil**

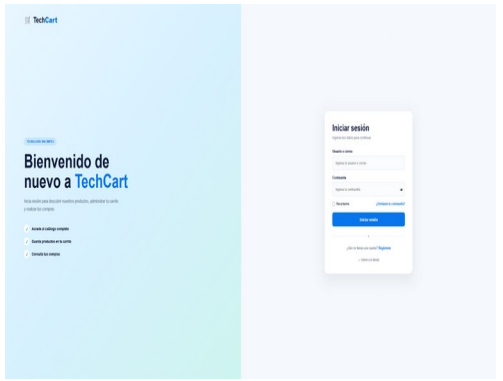
10. Diseño de interfaz - zona pública

La referencia visual se traduce a una identidad propia de TechCart: azul, turquesa, tarjetas limpias y jerarquía clara.

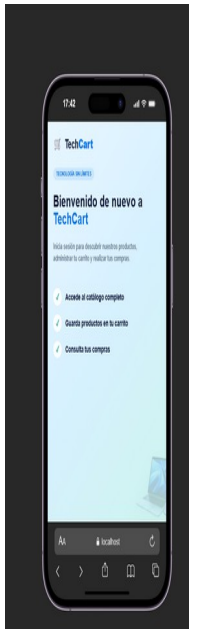
Landing / página pública



Login responsive



DESKTOP

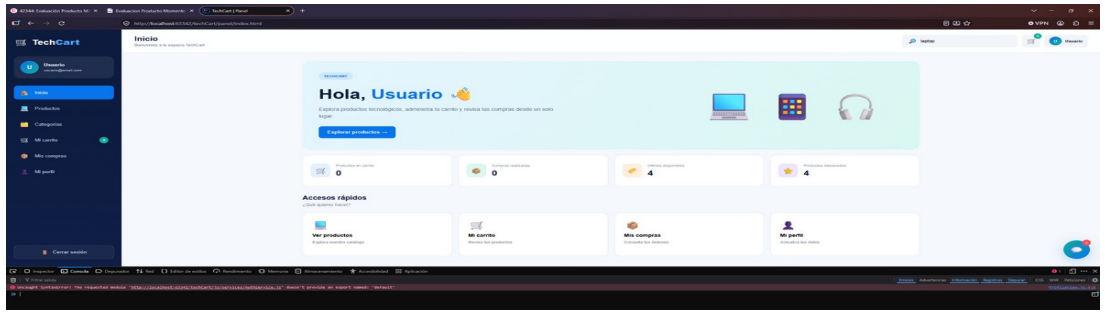


MÓVIL

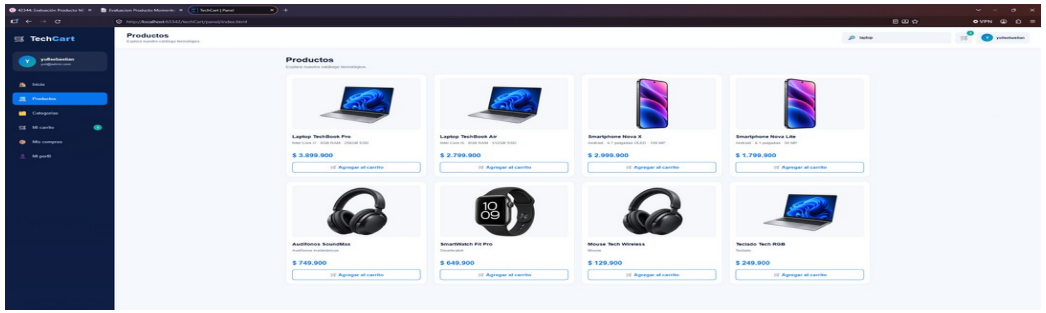
11. Diseño de interfaz - SPA interna

Sidebar, panel, catálogo y perfil comparten navegación y componentes visuales.

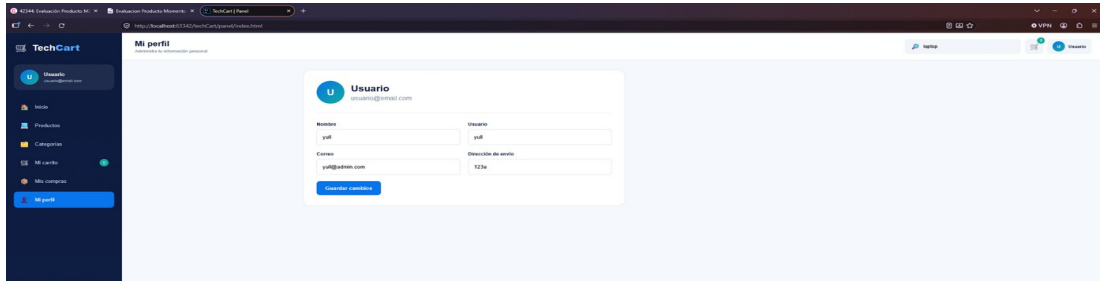
Inicio del panel



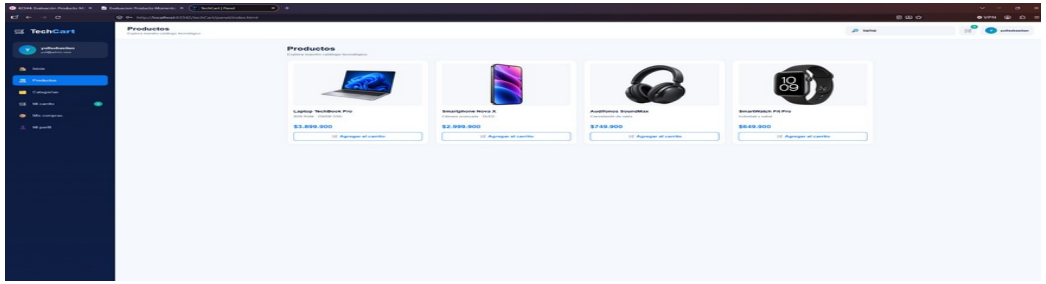
Catálogo



Perfil de usuario



Carrito y catálogo dinámico



12. Responsive y experiencia de usuario

La interfaz se adapta a escritorio, tablet y móvil; los componentes prioritarios cambian de disposición sin perder funcionalidad.

Móvil



Reglas de adaptación

- Sidebar → menú hamburguesa
- Grids de 4 columnas → 1/2 columnas
- Formularios → una columna
- Chatbot → ventana compacta
- Sin scroll horizontal accidental
- Botones táctiles y textos legibles

Criterio técnico

Media queries + layout flexible

El objetivo no es “encoger” el desktop, sino reorganizar la información según el espacio disponible y mantener visibles las acciones principales.

DESKTOP TABLET MÓVIL

13. Definición de terminado (Definition of Done)

Cada historia solo se considera cerrada cuando cumple funcionalidad, persistencia, interfaz y prueba básica.

✅ **Funcionalidad**

La historia cumple sus criterios de aceptación.

🧩 **POO**

La lógica se ubica en modelos/servicios y no se duplica.

💾 **Persistencia**

Los datos relevantes sobreviven a recargas cuando aplica.

🎨 **Interfaz**

SweetAlert2, estados vacíos y feedback visual correctos.

📱 **Responsive**

Desktop, tablet y móvil sin desbordes.

🧪 **Consola**

Sin errores JavaScript ni módulos faltantes.

🌿 **Git**

Commit descriptivo y cambios identificables.

📄 **Documentación**

Historia, UML y flujo actualizados.

TECHCART

De una interfaz atractiva a una aplicación estructurada

El proyecto integra historias de usuario verificables, sprints cortos, arquitectura modular, POO, persistencia local, SPA y diseño responsive en un flujo coherente de compra.

POO

SPA

JSON

LocalStorage

Responsive

Chatbot

LISTO PARA CONTINUAR CON PRUEBAS, ÓRDENES Y ENTREGA FINAL

AMPLIACIÓN DEL ALCANCE

Funcionalidades añadidas después del MVP original; todas operan en frontend con datos locales o simulados.

72 productos y catálogo

Ampliación de 18 a 72 artículos; búsqueda por texto, categorías y tarjetas con detalles.

Comparador

Selección de hasta 3 productos; filtros por tipo, tabla de atributos y agregar al carrito.

Reseñas y calificaciones

Estrellas 1–5, comentarios, promedios por producto y almacenamiento local.

Detalle de producto

Modal con imagen, características, stock, precio y acciones de compra.

Simulador de envíos

Destino, modalidad, tarifa y estimación; desglose asociado a la orden y PDF.

TechBot asíncrono

Consultas de envíos y pagos de prueba con Promesas y async/await; sin cobros reales.

HISTORIAS DE USUARIO NUEVAS · HU-11 A HU-15

Continuación del backlog original HU-01 a HU-10. Historias redactadas según funcionalidades observadas o descritas.

HU-11	**Comparar productos**	Como cliente, quiero comparar hasta tres productos de la misma familia para escoger con mejores criterios.
HU-12	**Reseñas**	Como cliente, quiero puntuar y comentar productos para compartir mi valoración y consultar promedios.
HU-13	**Ficha detallada**	Como cliente, quiero abrir una ficha con datos y acciones del producto sin perder la navegación.
HU-14	**Costo de envío**	Como cliente, quiero estimar tarifa y modalidad de envío y ver el total antes de confirmar.
HU-15	**TechBot asíncrono**	Como cliente, quiero consultar envíos y explorar medios de pago simulados sin bloquear la pantalla.

CRITERIOS DE ACEPTACIÓN · NUEVAS FUNCIONES

Criterios verificables para la demostración académica; validación integrada pendiente de pruebas del entorno final.

HU-11 · Comparador

Permite categoría y búsqueda; selecciona máximo 3; compara especificaciones compatibles; agrega la opción elegida al carrito.

HU-12 · Reseñas

Permite estrellas 1–5 y comentario; calcula promedio; actualiza en catálogo; almacena datos en navegador.

HU-13 · Detalles

Abre modal desde tarjeta; muestra descripción, especificaciones, stock y precio; permite comparar y agregar.

HU-14/15 · Envíos y bot

Cotiza según destino/modalidad; guarda desglose en orden y PDF; muestra pagos como simulación sin datos bancarios reales.

ARQUITECTURA MODULAR ACTUALIZADA

Separación de interfaz, servicios, modelos y persistencia. La asincronía no implica backend ni validación bancaria.

VISTAS / UI

PanelApp, Router, Sidebar
ProductsView, ProductDetailView
ComparadorView, ReviewsView
CartView, OrdersView, HistoryView
ProfileView, Chatbot, GlobalSearch

SERVICIOS

ProductService
ComparadorService
ResenaService
EnvioService
StorageService
AuthService, ClienteService

MODELOS / DATOS

Producto; Laptop; Smartphone
Accesorio; Cliente
CarritoCompra; Orden
ComparadorProductos; Resena
productos.json; localStorage

Regla UML: importación/uso ≠ herencia. La herencia JavaScript confirmada es Producto ← Laptop/Smartphone/Accesorio.

ASINCRONÍA APLICADA AL NEGOCIO

TechBot usa un flujo de consulta simulada; la UI continúa respondiendo mientras espera temporizadores y Promesas.

1 · Evento del usuario

Clic en Métodos de pago o Envíos

2 · Call Stack

Se inicia la función async; se presenta estado pendiente

3 · Web API / temporizador

setTimeout simula latencia; no consulta servicios bancarios

4 · Task Queue

Callback del temporizador se ejecuta cuando toca

5 · Microtask Queue

Se reanuda await al resolverse la Promesa

6 · Render

Se muestra estimación o estado de simulación

LIMITACIONES Y CONDICIONES DE DEMOSTRACIÓN

Precisiones necesarias al presentar TechCart en clase.

Qué SÍ se demuestra

POO: herencia y sobrescritura.
SPA y manipulación del DOM.
Persistencia local y órdenes PDF.
Búsqueda con debounce.
Promesas, async/await y Event Loop.
Tarifas y tiempos de envío simulados.

Qué NO debe afirmarse

No valida bancos ni descuenta dinero real.
No verifica inventario compartido.
No hay pasarela bancaria ni API logística.
localStorage no es una base remota.
Autenticación local no es seguridad productiva.

Evidencia de actualización: código compartido, capturas de la SPA y descripción del flujo final; no se ha auditado todo el repositorio.